StyleSense : Generative AI–Powered Fashion Recommendation System

Project Description:

StyleSense is a **Generative AI–based fashion recommendation system** designed to provide personalized outfit suggestions based on user preferences, occasion, budget, and optional clothing images. The system uses artificial intelligence and rule-based logic to analyze fashion inputs and generate styling advice, outfit recommendations, color analysis, and fashion trends.

The platform allows users to enter their style preferences, describe the occasion for which they need an outfit, select a budget range, and optionally upload an inspiration image. Using this information, the system processes the inputs through a backend AI engine and produces intelligent recommendations.

The application is implemented as a **web-based system using the Flask framework**. The frontend interface is built using **HTML, CSS, and JavaScript**, which is embedded directly inside the Python application. The backend communicates with **Hugging Face AI models** for image analysis and text generation.

If the AI models are unavailable or API requests fail, the system falls back to a **rule-based recommendation engine** that still provides meaningful styling suggestions. This ensures reliability and continuous functionality.

StyleSense aims to simplify fashion decision-making by providing **automated styling guidance and fashion insights**.

Scenarios:

Scenario 1: User Requests Outfit Recommendations Based on Style Preference

Description

In this scenario, a user wants fashion recommendations based on a preferred style without uploading an image.

Process

1. The user opens the Style Sense web application in a browser.
2. The user selects a **style preference** from the dropdown menu (e.g., Casual, Formal, Streetwear, Minimalist).
3. The user enters the **occasion**, such as "date night" or "office meeting".
4. The user specifies **color preferences** like pastel colors or monochrome.
5. The user selects a **budget range**.
6. The user clicks **“Get AI Recommendations.”**
7. The frontend sends the information to the **/api/recommendations endpoint**.

8. The backend processes the input and generates recommendations using the **fashion recommendation engine**.
9. The system returns the results to the user interface.

Output

The user receives:

- Outfit recommendations
- Styling tips
- Color suggestions
- Fashion trends
- Shopping suggestions

Scenario 2: User Uploads an Image for Fashion Analysis

Description

In this scenario, the user uploads an image of a clothing item or outfit to receive AI-generated fashion suggestions.

Process

1. The user selects style preferences and other inputs.
2. The user uploads an image using the **file upload feature**.
3. The image is converted into **Base64 format** in the frontend.
4. The image is sent to the backend API.
5. The backend calls the **BLIP image captioning model from Hugging Face**.
6. The model analyzes the image and produces a **text description of the outfit**.
7. The description is combined with user preferences.
8. The AI model generates personalized fashion recommendations.

Output

The system provides:

- Outfit suggestions that match the uploaded clothing item
- Styling tips
- Color compatibility analysis
- Current fashion trends

Scenario 3: Budget-Based Fashion Recommendation

Description

A user wants outfit suggestions that match a specific budget range.

Process

1. The user selects a **budget category** such as:

- Budget Friendly
- Moderate
- Premium
- Luxury

2. The user enters additional style information.
3. The request is sent to the backend.
4. The recommendation engine selects fashion suggestions suitable for the selected budget.

Output

The system provides shopping suggestions such as:

- Budget stores
- Designer brands
- Sustainable fashion options

Architecture Overview:

The StyleSense system follows a **three-layer architecture** that combines a user interface, a backend server, and artificial intelligence models to generate personalized fashion recommendations.

1. Presentation Layer (Frontend)

The frontend provides the user interface where users enter their fashion preferences and upload images. It is built using **HTML, CSS, and JavaScript** embedded in the Flask application. Users can select style preferences, specify occasions, choose budget ranges, and optionally upload clothing images. The frontend sends this information to the backend using **API requests**.

2. Application Layer (Backend)

The backend is implemented using the **Flask framework in Python**. It processes user inputs, manages image uploads, and communicates with AI services. The backend also contains API endpoints such as **/api/recommendations** for generating outfit suggestions and **/api/trends** for retrieving fashion trends.

3. AI Processing Layer

This layer performs the core intelligence of the system. It uses **Hugging Face AI models** to analyze images and generate fashion recommendations. The **BLIP image captioning model** analyzes uploaded images, while a **text generation model (GPT-2)** produces styling advice and outfit suggestions. If the AI models are unavailable, a **rule-based fallback system** generates recommendations.

Data Flow

The user enters preferences or uploads an image → the frontend sends data to the Flask backend → the backend processes inputs and calls AI models → recommendations are generated → results are returned to the user interface.

If you want, I can also give you **brief versions of the next sections like Core Components, AI Models Used, and Workflow**, so your documentation stays **consistent and concise**.

Pre-Requisites:

The following requirements must be met before running the StyleSense: Generative AI–Powered Fashion Recommendation System.

1. Programming Language

- **Python 3.8 or higher**

2. Required Libraries

- **Flask – for building the backend web application**
- **Flask-CORS – for enabling cross-origin requests**
- **Requests – for communicating with external APIs**
- **Werkzeug – for secure file handling**

3. AI Service Access

- **A Hugging Face API Key is required to access AI models for image analysis and text generation.**

4. Web Browser

- **Any modern browser such as Chrome, Edge, or Firefox to run the web interface.**

5. Internet Connection

- **Required to access Hugging Face AI APIs for generating fashion recommendations.**

Tools & Frameworks:

The **StyleSense: Generative AI–Powered Fashion Recommendation System** is developed using several tools and frameworks that support web development and AI integration. **Python** is used as the main programming language to implement the backend logic and handle data processing. The **Flask framework** is used to build the web server and manage API requests between the frontend and backend. **Flask-CORS** enables secure communication between different components of the application. The system integrates **Hugging Face APIs** to access AI models such as BLIP for image captioning and GPT-2 for generating fashion recommendations. For the user interface, **HTML, CSS, and JavaScript** are used to create an interactive web application. Additionally, the **Requests library** is used to send HTTP requests to external AI services, and **Werkzeug** is used for secure handling of uploaded image files. These tools together help build an efficient and scalable AI-based fashion recommendation platform.

Project Workflow:

Phase 1: Environment Setup

The required development environment is prepared to build and run the **StyleSense: Generative AI–Powered Fashion Recommendation System**. First, **Python 3.8 or higher** is installed as the main programming language for developing the backend application. After installing Python, the necessary libraries such as **Flask, Flask-CORS, Requests, and Werkzeug** are installed using the pip package manager. A project directory is then created to store the application files, including the main Python script and upload folder for storing images. Next, a **Hugging Face API key** is configured as an environment variable to enable access to AI models for image analysis and text generation. Finally, the Flask server is started, and the application is accessed through a web browser using **localhost**, ensuring that the development environment is properly set up before proceeding to the next phases.

Phase 2: Backend Development

The backend of the **StyleSense system** is developed using the **Flask framework in Python**. The backend is responsible for handling user requests, processing input data, and generating fashion recommendations. API routes such as /, **/api/recommendations**, and **/api/trends** are created to manage communication between the frontend and backend. The backend also processes form data including style preference, occasion, colors, budget, and optional image uploads. If an image is provided, it is analyzed using the **Hugging Face BLIP image captioning model** to generate a description of the outfit. Based on the user inputs and image analysis, the system generates fashion recommendations using AI models or a rule-based fallback system. The backend then returns the generated results as **JSON responses** to the frontend for display.

Phase 3: Frontend Development

The **user interface of the StyleSense system** is developed using **HTML, CSS, and JavaScript**. The frontend is responsible for collecting user inputs such as style preference, occasion, color preferences, budget range, and optional image uploads. A responsive interface is designed to allow users to easily interact with the system and request fashion recommendations. JavaScript is used to handle user actions, process image uploads, and send the collected data to the backend through API requests. After receiving the response from the backend, the frontend dynamically displays the generated **outfit recommendations, styling tips, color analysis, and fashion trends** to the user in a clear and interactive format.

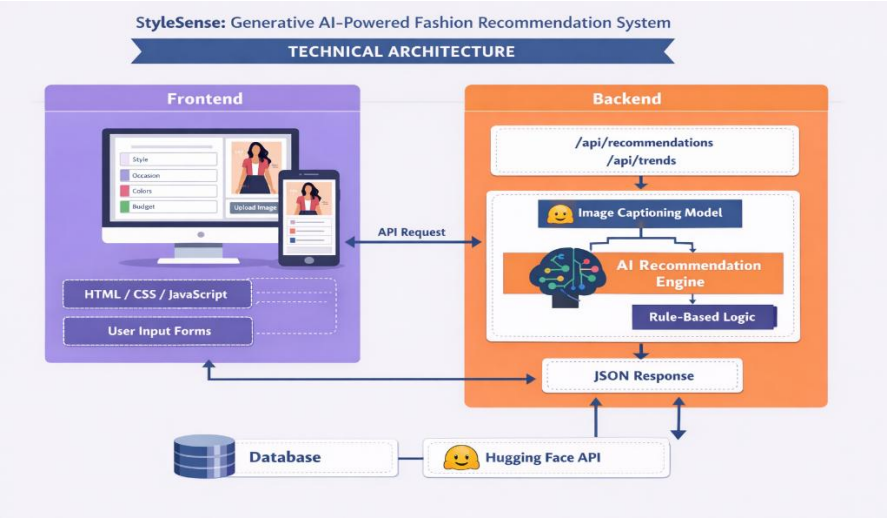
Phase 4:

The StyleSense application is deployed and made ready for use. The Flask backend server is started, and the application is hosted so that users can access it through a web browser. The required environment variables such as the Hugging Face API key are configured to enable AI model access. All dependencies and required libraries are installed to ensure the system runs smoothly. After deployment, the application is tested to verify that the frontend interface, backend APIs, image analysis, and recommendation generation work correctly. Once testing is successful, the StyleSense platform becomes available for users to generate personalized fashion recommendations.Phase

5: Testing

The StyleSense system is tested to ensure that all components work correctly. The application is checked by providing different user inputs such as style preference, occasion, color preferences, and budget to verify that the system generates accurate fashion recommendations. The image upload feature is also tested to confirm that the system correctly analyzes images and produces relevant styling suggestions. Additionally, the API endpoints are tested to ensure proper communication between the frontend and backend. Any errors or issues identified during testing are fixed to improve the performance, reliability, and usability of the system before final use.

Technical Architecture:



MILESTONE 1: Environment Setup & AI Model Configuration

This milestone establishes the **core AI environment for StyleSense** by installing required dependencies and configuring access to the **Hugging Face AI models** used for image captioning and fashion recommendation generation.

Completing this milestone ensures that the **backend application can communicate with AI models** to analyze fashion images and generate personalized styling recommendations

Activity 1.1 Install Python Environment & Dependencie

Install **Python 3.8 or higher** from the official website.

Download from:
https://www.python.org/downloads/

Create and activate a **virtual environment** to manage project dependencies.

For Windows

Create virtual environment:

python -m venv venv

Activate virtual environment:

venv\Scripts\activate

Install required project dependencies:

pip install flask flask-cors requests python-dotenv

These libraries are required for:

- Flask → Web backend development
- Flask-CORS → Allow frontend-backend communication
- Requests → Sending API requests to AI services
- Python-dotenv → Managing environment variables

Verification

Check Python version:

python --version

Check installed packages:

pip list

If the environment is correctly configured, the list will include **flask, flask-cors, requests, and python-dotenv**.

.

Activity 1.2: Configure Hugging Face API

StyleSense uses **Hugging Face AI models** for image analysis and text-based fashion recommendations.

Steps

1. Create a **Hugging Face account**

https://huggingface.co

2. Generate an **API Access Token**

Profile → Settings → Access Tokens → Create Token

3. Add the API key as an **environment variable**

Example:

export HF_API_KEY="your_huggingface_api_key"

For Windows:

set HF_API_KEY=your_huggingface_api_key

This API key allows the backend to communicate with Hugging Face models.

MILESTONE 2: Backend Development

This milestone focuses on building the backend architecture for the StyleSense system using the Flask framework in Python. The backend is responsible for processing user inputs, interacting with AI models, and generating personalized fashion recommendations. It acts as the central component that connects the frontend interface with the AI services used for image analysis and recommendation generation.

The backend receives requests from the user interface, processes data such as style preferences, occasion, color preferences, budget, and optional image uploads, and communicates with Hugging Face AI models to generate fashion insights. The generated recommendations are then returned to the frontend in JSON format for display.

Activity 2.1 Flask Application Setup

The backend application is developed using **Flask**, a lightweight Python web framework.

The main application file:

stylesense_app.py

The Flask app is initialized as follows:

```
from flask import Flask
app = Flask(__name__)
```

Additional configurations include:

- Enabling **CORS** for frontend communication
- Setting **maximum upload size**
- Creating an **uploads directory** for storing images

Example configuration from the code:

```
app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024
app.config['UPLOAD_FOLDER'] = 'uploads'
```

Activity 2.2: API Endpoint Development

The backend contains several API endpoints that process requests and return results.

1. Home Endpoint

/

This endpoint loads the **StyleSense web interface** that contains the fashion recommendation form.

Example route:

```
@app.route('/')
def index():
    return render_template_string(HTML_TEMPLATE)
```

2. Recommendation API

/api/recommendations

This API receives user inputs and generates fashion recommendations.

Inputs received from frontend:

- Style preference
- Occasion
- Color preferences
- Budget range
- Additional notes
- Optional uploaded image

Example code:

```
@app.route('/api/recommendations', methods=['POST'])
def get_recommendations():
```

The backend processes the data and returns results such as:

- Outfit recommendations
- Styling tips
- Color analysis
- Fashion trends
- Shopping suggestions

3. Fashion Trends API

/api/trends

This endpoint returns **current fashion trends and seasonal styles**.

Example response:

```
{
 "2024_trends": [
 "Sustainable Fashion",
 "Gender-Neutral Clothing",
 "Vintage Revival"
 ]
}
```

Activity 2.3 AI Service Integration

The backend integrates **Hugging Face AI models** to analyze fashion images and generate recommendations.

Image Captioning Model

The system uses the **BLIP image captioning model**:

Salesforce/blip-image-captioning-large

This model analyzes uploaded images and generates descriptions.

Example from the code:

```
API_URL = "https://api-inference.huggingface.co/models/Salesforce/blip-image-captioning-large"
```

The generated description helps the system understand clothing items.

Text Generation Model

The system uses **GPT-2 text generation** to produce styling recommendations.

Example model:

gpt2

Example code from the backend:

API_URL = "https://api-inference.huggingface.co/models/gpt2"

This model generates fashion advice based on user preferences.

Activity 2.4 Fallback Recommendation System

To ensure system reliability, the backend includes a **rule-based fallback system**.

If the AI API fails, the system generates recommendations using predefined fashion knowledge.

The fallback system supports multiple styles:

- Casual
- Formal
- Business
- Streetwear
- Vintage
- Minimalist
- Bohemian
- Athletic

Each style contains:

- Outfit suggestions
- Styling tips
- Color recommendations

Activity 2.5 JSON Response Generation API Endpoint

After generating recommendations, the backend sends the results to the frontend in **JSON format**.

Example output:

```
{
 "outfit_recommendations": "...",
 "styling_tips": "...",
 "color_analysis": "...",
 "trends": "...",
 "trend_tags": ["#StyleTips", "#Trending"],
```

"shopping_suggestions": "..."
}

The frontend then displays these recommendations to the user.

MILESTONE 3: Frontend Development

This milestone focuses on developing the **user interface of the StyleSense system** that allows users to interact with the application and request personalized fashion recommendations. The frontend is implemented using **HTML, CSS, and JavaScript**, which are embedded within the Flask application. It provides a responsive and user-friendly interface where users can input their fashion preferences and upload images.

Main files:

- stylesense_app.py
- uploads/
- .env
- requirements.txt

Features:

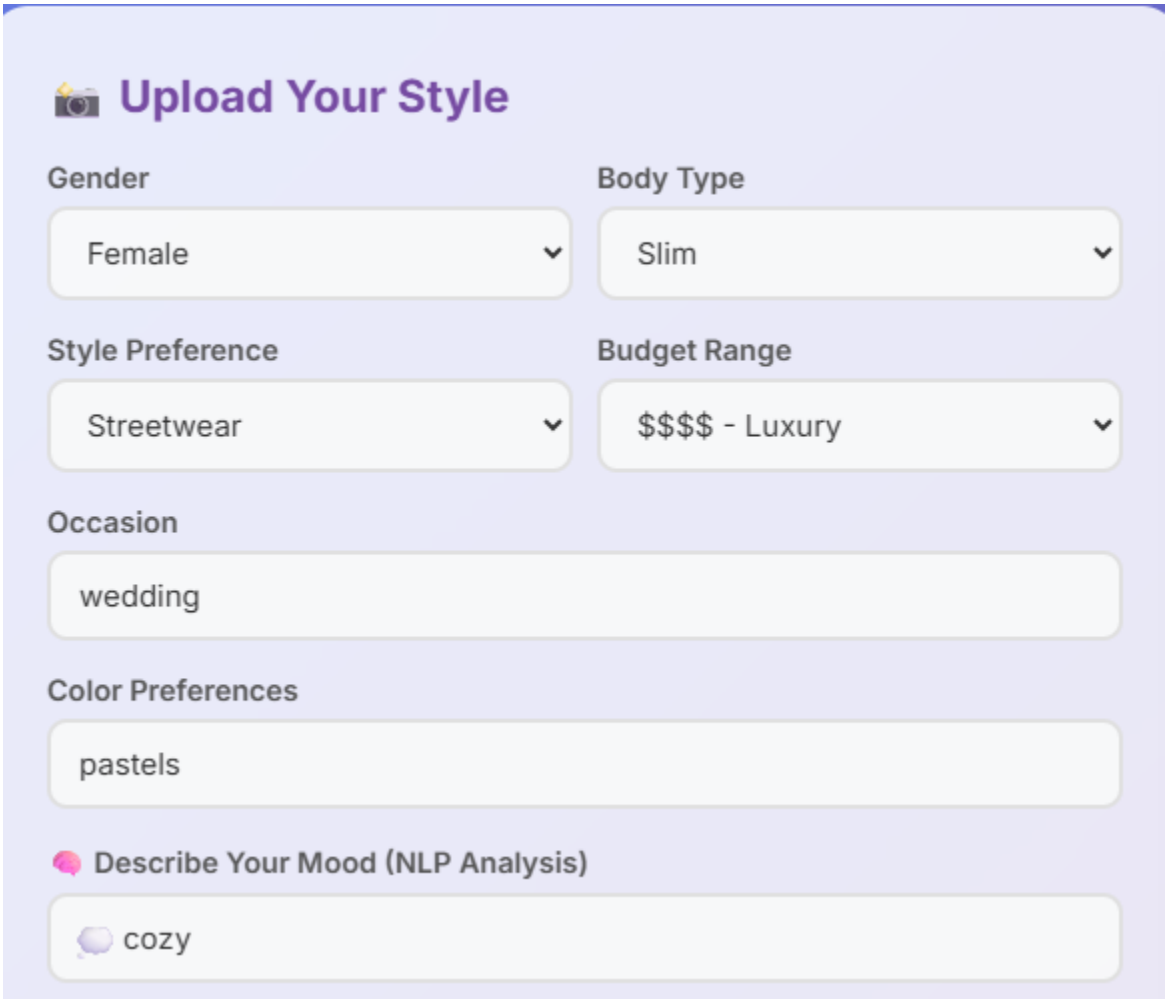
- **Personalized Fashion Recommendations based on user preferences such as style, occasion, colors, and budget.**
- **Image-Based Fashion Analysis where users can upload images to receive AI-generated styling suggestions.**
- **AI-Powered Recommendation System using Hugging Face models for image captioning and text generation.**
- **Styling Tips and Color Analysis to help users improve their outfit choices.**
- **Current Fashion Trends Display to keep users updated with trending styles.**
- **Budget-Based Shopping Suggestions for different price ranges.**

MILESTONE 4: Deployment

The **StyleSense application** is deployed by running the Flask backend server on a local machine or hosting platform. All required dependencies are installed, and the **Hugging Face API key** is configured as an environment variable to enable AI model access. Once the server is started, the application can be accessed through a web browser using **localhost**. The deployed system allows users to interact with the interface, upload images, and receive AI-powered fashion recommendations in real time.

MILESTONE 5: Testing

The StyleSense system is tested to ensure that all components function correctly. Different user inputs such as style preference, occasion, color preferences, and budget are provided to verify that the system generates accurate fashion recommendations. The image upload feature is also tested to confirm that images are correctly processed and analyzed by the AI model. Additionally, the API endpoints are checked to ensure proper communication between the frontend and backend. Any errors identified during testing are fixed to improve the system's performance and reliability.



SMARTBRIDGE
Let's Bridge the Gap

Upload Inspiration Image (Optional)

Click to upload or drag and drop
JPG, PNG up to 10MB

Additional Notes

indian style

✨ Get AI Recommendations

Your Personalized Recommendations

Download | Copy

Visual Representation

🎯 Outfit Recommendations

For a luxury streetwear-infused Indian wedding look, consider a heavily embellished, dark-toned lehenga choli with a modern silhouette. Opt for a cropped choli with intricate embroidery or beadwork, paired with a high-waisted lehenga skirt featuring asymmetrical cuts or contemporary detailing. Accessorize with statement sneakers or embellished chunky heels instead of traditional sandals for a unique twist.

💡 Styling Tips

Embrace layering to create depth and visual interest. A long, sheer jacket or an oversized, dark-toned bomber jacket can be draped over the lehenga to add a streetwear element. Play with texture by incorporating fabrics like velvet, leather, and silk into the overall look to create a visually appealing contrast, while ensuring it complements the slim body type.

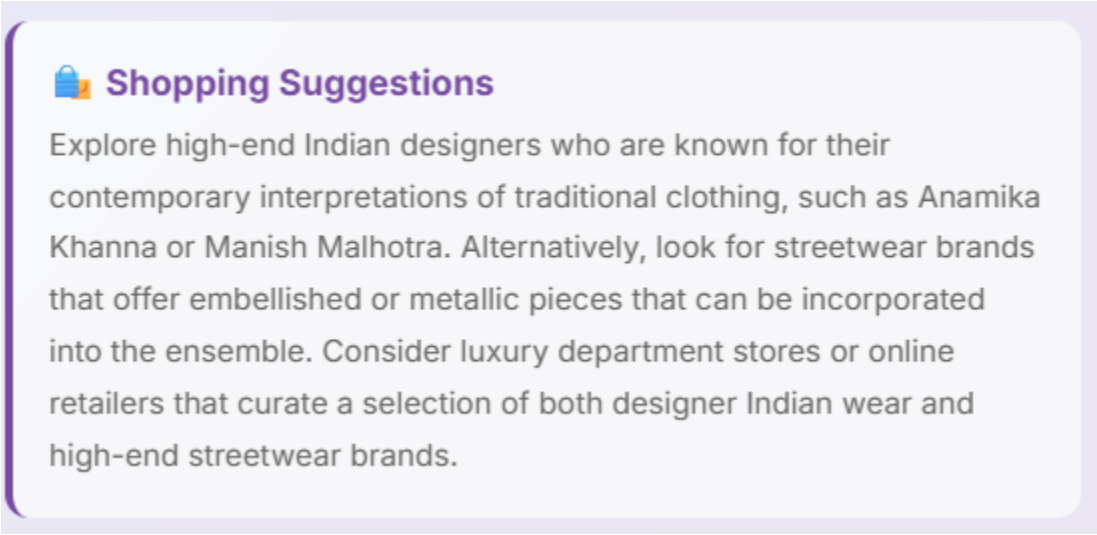
🎨 Color Analysis

Given the preference for dark colors, explore shades of deep jewel tones like emerald green, sapphire blue, or ruby red, alongside classic black or charcoal gray. These colors will complement a slim figure and create a sophisticated, yet edgy look. Consider incorporating metallic accents like gold or silver through embellishments, embroidery, or accessories to add a touch of luxury and visual impact.

📈 Current Trends

The current trends influencing this look involve the fusion of cultural aesthetics with modern streetwear silhouettes. Key trends include embellished athleisure wear, deconstructed traditional garments, and the incorporation of metallic accents and bold color blocking. Luxury brands are increasingly experimenting with unexpected collaborations and unconventional materials, making them ideal choices for a luxury streetwear vibe.

Indian Fusion | Embellished Streetwear | Dark Jewel Tones | Deconstructed Lehenga



Conclusion:=

The **StyleSense: Generative AI–Powered Fashion Recommendation System** successfully demonstrates how artificial intelligence can be used to provide personalized fashion suggestions. The system combines a user-friendly frontend interface with a Flask-based backend and AI models to analyze user preferences and optional images. By using AI-powered recommendation techniques, the platform generates outfit suggestions, styling tips, color analysis, and fashion trend insights. The system improves the user experience by helping individuals choose suitable outfits based on their style, occasion, and budget. Overall, StyleSense shows the potential of integrating **web technologies and AI models** to create an intelligent fashion recommendation platform